



Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)

ДИПЛОМНЫЙ ПРОЕКТ

по основной образовательной программе

среднего профессионального образования по специальности

09.02.07 «Информационные системы и программирование» на тему:

Разработка базы данных для «Сервиса по продаже внутриигровых предметов и
валюты»

Выполнил студент	_____	_____
	(подпись)	(фамилия, имя, отчество)
Руководитель	_____	_____
	(подпись)	(ученая степень, фамилия, имя, отчество)
Рецензент	_____	_____
	(подпись)	(ученая степень, фамилия, имя, отчество)

		(должность)

		(место работы)
Председатель ГЭК	_____	_____
	(подпись)	(ученая степень, фамилия, имя, отчество)
Оценка	_____	_____
	(прописью)	Дата защиты: __.__.20__г.

Тверь, 2026 г.

Содержание

Содержание	2
Введение	4
1. Теоретические основы разработки базы данных для сервиса по продаже внутриигровых предметов и валюты	7
1.1. Особенности современных сервисов по продаже внутриигровых предметов и валюты	7
1.2. Основы проектирования баз данных.....	8
1.3. Использование PostgreSQL в качестве основной системы управления базами данных	10
1.4. Анализ средств объектно-реляционного отображения и обоснование выбора Sequelize ORM.....	12
1.5. Использование Redis для кэширования данных	15
1.6. Использование Elasticsearch для поиска и аналитики	16
1.7. Архитектурные особенности высоконагруженных систем	17
1.8. Анализ требований к базе данных маркетплейса игровых товаров	18
1.9. Моделирование сущностей и связей базы данных.....	20
2. Проектирование и разработка базы данных сервиса по продаже внутриигровых предметов и валюты.....	22
2.1. Анализ требований к разрабатываемой базе данных	22
2.2. Проектирование основных сущностей базы данных	24
2.3. Разработка логической модели базы данных.....	40
2.4. Разработка физической модели базы данных PostgreSQL	43
2.5. Реализация моделей базы данных средствами Sequelize ORM	47
2.6. Реализация кэширования данных с использованием Redis	50
2.7. Реализация системы поиска логов на базе Elasticsearch	52
3. Тестирование базы данных и разработанного решения.....	54
3.1. Организация процесса тестирования и его цели	54

3.2. Тестирование реляционной модели данных и целостности PostgreSQL	56
3.3. Тестирование корректности и атомарности финансовых операций	57
3.4. Тестирование Redis (кэширования данных)	59
Заключение	61
Список использованных источников	65

Введение

В современном цифровом обществе индустрия видеоигр занимает одно из ведущих мест среди направлений информационных технологий и электронной коммерции. Стремительное развитие многопользовательских онлайн-игр, киберспорта и внутриигровых сервисов привело к формированию полноценного рынка виртуальных товаров, включающего игровые предметы, аккаунты, внутриигровую валюту и различные цифровые услуги. Пользователи активно совершают операции купли-продажи виртуальных объектов, а игровые платформы и маркетплейсы становятся важным элементом цифровой экономики.

Актуальность темы дипломной работы обусловлена необходимостью создания надежной, масштабируемой и производительной базы данных для сервиса по продаже внутриигровых предметов и валюты. Современные маркетплейсы цифровых товаров должны обеспечивать высокую скорость обработки запросов, безопасное хранение пользовательских данных, поддержку большого количества одновременных операций и возможность интеграции с различными внешними сервисами. При этом особое значение приобретает правильное проектирование структуры базы данных, поскольку именно она является основой функционирования всей серверной части системы.

Дополнительным фактором актуальности является постоянный рост количества пользователей игровых сервисов и объемов совершаемых транзакций. Некачественно спроектированная база данных может привести к снижению производительности системы, потере данных, возникновению ошибок при обработке заказов и ухудшению пользовательского опыта. В связи с этим разработка эффективной базы данных для маркетплейса игровых товаров представляет собой важную практическую задачу в области проектирования информационных систем.

Целью дипломной работы является разработка базы данных для сервиса по продаже внутриигровых предметов и валюты, обеспечивающей надежное хранение

данных, эффективную обработку запросов и поддержку основных бизнес-процессов маркетплейса.

Для достижения поставленной цели необходимо решить следующие задачи:

- Рассмотреть теоретические аспекты проектирования баз данных и современных маркетплейсов цифровых товаров.

- Проанализировать особенности хранения и обработки данных в сервисах электронной коммерции.

- Исследовать предметную область сервиса по продаже внутриигровых предметов и валюты.

- Выполнить проектирование структуры базы данных с учетом требований масштабируемости и производительности.

- Разработать логическую и физическую модели базы данных.

- Реализовать базу данных с использованием системы управления базами данных PostgreSQL.

- Организовать взаимодействие базы данных с серверной частью приложения на основе NestJS и GraphQL.

- Реализовать механизмы кэширования и обработки сообщений с использованием Redis и Kafka.

- Выполнить тестирование разработанной базы данных и оценить эффективность ее работы.

- Провести анализ полученных результатов и сформулировать рекомендации по дальнейшему развитию системы.

Объектом исследования является сервис по продаже внутриигровых предметов и валюты.

Предметом исследования выступают методы проектирования и реализации базы данных для высоконагруженного маркетплейса цифровых игровых товаров.

Практическая значимость работы заключается в разработке базы данных, которая может быть использована в реальных проектах электронной коммерции, связанных с продажей игровых предметов и внутриигровой валюты. Разработанная

система обеспечивает эффективное хранение данных, поддержку масштабирования, обработку большого количества пользовательских запросов и интеграцию с современными серверными технологиями.

При выполнении дипломной работы использовались технологии и инструменты: PostgreSQL, NestJS, GraphQL, Redis, Kafka, Elasticsearch, Kibana, MinIO, Docker и Prometheus. Использование данного стека технологий позволяет создать современную распределенную систему с высокой производительностью, отказоустойчивостью и возможностью мониторинга состояния сервисов.

1. Теоретические основы разработки базы данных для сервиса по продаже внутриигровых предметов и валюты

1.1. Особенности современных сервисов по продаже внутриигровых предметов и валюты

В настоящее время игровая индустрия является одной из наиболее быстроразвивающихся отраслей информационных технологий. Развитие многопользовательских онлайн-игр, цифровых торговых площадок и систем внутриигровых покупок привело к формированию полноценного рынка виртуальных товаров. Пользователи приобретают игровые предметы, внутриигровую валюту, скины, оружие, экипировку, аккаунты и другие цифровые объекты как внутри самих игр, так и на специализированных внешних маркетплейсах.

Сервисы по продаже внутриигровых предметов представляют собой сложные информационные системы, обеспечивающие взаимодействие между продавцами и покупателями цифровых товаров. Основными функциями подобных сервисов являются:

- регистрация и авторизация пользователей;
- хранение информации о товарах;
- обработка заказов и платежей;
- управление каталогом игровых предметов;
- обеспечение поиска и фильтрации товаров;
- хранение истории операций;
- обработка пользовательских уведомлений;
- аналитика и мониторинг активности системы.

Особенностью подобных платформ является высокая нагрузка на серверную часть и базу данных. Большое количество одновременно подключенных пользователей, постоянное обновление информации о товарах и проведение

транзакций требуют использования производительных и масштабируемых решений.

Кроме того, сервисы цифровой торговли предъявляют повышенные требования к надежности хранения данных. Потеря информации о заказах, пользователях или транзакциях может привести к финансовым потерям и снижению доверия пользователей к платформе. Поэтому при проектировании базы данных необходимо учитывать механизмы резервирования, обеспечения целостности данных и оптимизации производительности.

Дополнительной особенностью маркетплейсов игровых товаров является необходимость работы с большим объемом слабо структурированных данных. Например, игровые предметы могут обладать различными характеристиками в зависимости от конкретной игры: уровнем редкости, типом оружия, модификациями, качеством, сервером или регионом использования. Это требует гибкого подхода к хранению данных и применения современных методов проектирования баз данных.

Современные сервисы также активно используют микросервисную архитектуру, позволяющую разделить функциональность системы на независимые компоненты. Такой подход обеспечивает возможность горизонтального масштабирования и упрощает сопровождение программного обеспечения. При этом база данных становится ключевым элементом всей системы, обеспечивая взаимодействие между различными сервисами.

1.2. Основы проектирования баз данных

База данных представляет собой организованную совокупность взаимосвязанных данных, предназначенных для долговременного хранения и эффективной обработки. Основной задачей базы данных является обеспечение надежного хранения информации, быстрого доступа к данным и поддержки операций изменения информации.

Проектирование базы данных включает несколько основных этапов:

- анализ предметной области;
- построение концептуальной модели;
- разработка логической модели;
- создание физической модели;
- оптимизация структуры хранения данных.

На этапе анализа предметной области определяются сущности системы, их атрибуты и связи между ними. Для сервиса продажи внутриигровых предметов основными сущностями являются:

- пользователи;
- товары;
- категории;
- игровые проекты;
- заказы;
- платежи;
- отзывы;
- уведомления;
- транзакции.

Концептуальная модель базы данных обычно строится с использованием ER-диаграмм. Данный подход позволяет визуально представить структуру системы и взаимосвязи между сущностями.

После построения концептуальной модели выполняется переход к логическому проектированию. На данном этапе определяются таблицы, поля, типы данных, первичные и внешние ключи.

Пример логической структуры таблицы пользователей приведен в таблице 1. Таблица 1 – Пример логической структуры таблицы пользователей.

Поле	Тип данных	Назначение
1	2	3
id	UUID	Уникальный идентификатор пользователя

1	2	3
email	VARCHAR	Адрес электронной почты
password_hash	VARCHAR	Хэш пароля
username	VARCHAR	Имя пользователя
balance	NUMERIC	Баланс пользователя
created_at	TIMESTAMP	Дата регистрации

При проектировании базы данных особое внимание уделяется нормализации данных. Нормализация позволяет уменьшить избыточность информации и предотвратить появление аномалий при обновлении данных.

Основными нормальными формами являются:

- первая нормальная форма;
- вторая нормальная форма;
- третья нормальная форма.

Третья нормальная форма обеспечивает устранение транзитивных зависимостей и считается оптимальной для большинства информационных систем.

Однако в высоконагруженных системах иногда применяется частичная денормализация данных. Такой подход позволяет повысить производительность запросов за счет хранения заранее агрегированной информации.

1.3. Использование PostgreSQL в качестве основной системы управления базами данных

Для реализации базы данных сервиса по продаже внутриигровых предметов используется система управления базами данных PostgreSQL. Данная СУБД является одной из наиболее популярных реляционных систем благодаря высокой производительности, надежности и поддержке современных возможностей.

Основными преимуществами PostgreSQL являются:

- поддержка стандарта SQL;
- работа с транзакциями;
- обеспечение целостности данных;
- поддержка JSON и JSONB;
- возможность горизонтального масштабирования;
- наличие механизмов репликации;
- поддержка полнотекстового поиска;
- высокая безопасность.

Одной из важных особенностей PostgreSQL является поддержка типа JSONB, позволяющего хранить полуструктурированные данные. Это особенно важно для игровых маркетплейсов, где различные категории товаров могут содержать уникальные наборы характеристик.

Использование JSONB позволяет обеспечить гибкость структуры базы данных без необходимости создания большого количества дополнительных таблиц.

Также PostgreSQL поддерживает индексацию JSON-полей, что существенно ускоряет выполнение поисковых запросов.

Для обеспечения производительности активно используются следующие типы индексов:

- B-Tree;
- Hash;
- GIN;
- GiST.

GIN-индексы особенно эффективны при работе с JSONB и полнотекстовым поиском.

Дополнительным преимуществом PostgreSQL является поддержка ACID-транзакций. Это обеспечивает корректность выполнения финансовых операций и предотвращает потерю данных при возникновении ошибок.

1.4. Анализ средств объектно-реляционного отображения и обоснование выбора Sequelize ORM

При разработке современных серверных приложений взаимодействие с базой данных чаще всего осуществляется с использованием технологий объектно-реляционного отображения (ORM — Object Relational Mapping). ORM позволяет связать объекты языка программирования с таблицами реляционной базы данных и автоматизировать выполнение большинства операций по работе с данными.

Использование ORM существенно упрощает процесс разработки программного обеспечения, поскольку разработчику не требуется вручную формировать большое количество SQL-запросов. Вместо этого работа с данными осуществляется через объекты и модели предметной области.

Основными преимуществами ORM являются:

- уменьшение объема программного кода;
- повышение читаемости проекта;
- поддержка миграций базы данных;
- снижение вероятности SQL-инъекций;
- обеспечение типизации данных;
- упрощение сопровождения программного обеспечения;
- независимость прикладной логики от конкретной СУБД.

Для разработки серверной части маркетплейса игровых товаров на платформе Node.js существует несколько популярных ORM-решений:

- Sequelize;
- TypeORM;
- Prisma;
- MikroORM.

Рассмотрим основные особенности каждого инструмента.

Sequelize является одной из наиболее распространенных ORM для Node.js и поддерживает работу с PostgreSQL, MySQL, MariaDB, SQLite и Microsoft SQL Server.

Основные возможности Sequelize:

- поддержка связей между сущностями;
- работа с транзакциями;
- миграции базы данных;
- механизмы валидации данных;
- поддержка сложных SQL-запросов;
- возможность выполнения нативного SQL.

Преимуществом Sequelize является высокая зрелость проекта, большое сообщество разработчиков и наличие большого количества готовых решений для интеграции с NestJS.

TypeORM является одной из наиболее популярных ORM среди разработчиков приложений на TypeScript.

Преимущества TypeORM:

- удобная интеграция с NestJS;
- развитая система декораторов;
- поддержка Active Record и Data Mapper;
- хорошая документация.

Недостатками являются:

- относительно низкая производительность при работе со сложными запросами;
- большое количество проблем совместимости между версиями;
- сложность оптимизации высоконагруженных приложений.

Для крупных проектов с большим количеством транзакций TypeORM может создавать дополнительную нагрузку на систему.

Prisma представляет собой современную ORM нового поколения, ориентированную на строгую типизацию и автоматическую генерацию кода.

Основные преимущества Prisma:

- высокий уровень типобезопасности;
- удобный интерфейс разработки;
- автоматическая генерация клиентского кода;
- качественная документация.

Однако использование Prisma сопровождается рядом ограничений:

- менее гибкая работа со сложными SQL-конструкциями;
- дополнительные накладные расходы на генерацию клиента;
- ограниченные возможности при реализации некоторых нестандартных сценариев.

Кроме того, Prisma наиболее эффективно показывает себя в проектах, разрабатываемых с нуля, тогда как существующая архитектура рассматриваемого проекта уже построена на Sequelize.

MikroORM является сравнительно молодой ORM, ориентированной на TypeScript.

Преимущества:

- высокая производительность;
- поддержка Unit of Work;
- современная архитектура.

Недостатки:

- небольшое сообщество;
- меньшее количество примеров и готовых решений;
- меньшая распространенность в корпоративных проектах.

По этой причине использование MikroORM может увеличить затраты на сопровождение проекта.

На основании проведенного анализа можно сделать вывод, что Sequelize является наиболее подходящим решением для реализации базы данных маркетплейса игровых товаров.

В рамках разработки сервиса по продаже внутриигровых предметов и валюты в качестве ORM выбрана Sequelize.

Во-первых, архитектура существующего проекта уже реализована на основе Sequelize, что исключает необходимость перехода на другую ORM и переработки значительной части программного кода.

Во-вторых, Sequelize предоставляет широкие возможности по работе со сложными запросами PostgreSQL. Для маркетплейса игровых товаров характерно выполнение большого количества операций фильтрации, сортировки и агрегации данных, поэтому возможность использования нативных SQL-запросов является важным преимуществом.

В-третьих, ORM поддерживает механизмы транзакций, которые необходимы для корректной обработки финансовых операций, оформления заказов и изменения баланса пользователей.

В-четвертых, Sequelize хорошо интегрируется с фреймворком NestJS, используемым при разработке серверной части проекта, что упрощает построение модульной архитектуры приложения.

Кроме того, данное решение обладает высокой степенью зрелости, активно используется в промышленных проектах и имеет большое сообщество разработчиков, что обеспечивает наличие подробной документации и большого количества готовых примеров.

Таким образом, использование Sequelize позволяет обеспечить удобное взаимодействие с базой данных PostgreSQL, повысить скорость разработки серверной части приложения и сохранить возможность дальнейшего масштабирования системы.

1.5. Использование Redis для кэширования данных

Одной из основных проблем высоконагруженных систем является увеличение времени ответа базы данных при большом количестве запросов. Для решения данной проблемы применяется технология кэширования.

Redis представляет собой высокопроизводительное хранилище данных типа «ключ-значение», размещающее данные в оперативной памяти.

Основными задачами Redis в проекте являются:

- кэширование популярных товаров;
- хранение пользовательских сессий;
- ускорение обработки запросов;
- хранение временных токенов;
- ограничение частоты запросов;
- обработка очередей сообщений.

Применение Redis позволяет существенно снизить нагрузку на PostgreSQL и повысить скорость работы системы.

При повторном запросе данные будут получены из кэша без обращения к основной базе данных.

Дополнительно Redis поддерживает механизм TTL (Time To Live), позволяющий автоматически удалять устаревшие данные.

Использование Redis особенно эффективно для:

- frequently requested data;
- пользовательских корзин;
- хранения JWT-токенов;
- временных уведомлений;
- лимитирования API-запросов.

1.6. Использование Elasticsearch для поиска и аналитики

Для реализации быстрого полнотекстового поиска в системе используется Elasticsearch.

Стандартные SQL-запросы PostgreSQL подходят для структурированных данных, однако при большом объеме текстовой информации и сложных поисковых

условиях их производительность может снижаться. Elasticsearch решает данную проблему за счет использования инвертированных индексов.

Основные возможности Elasticsearch:

- полнотекстовый поиск;
- фильтрация данных;
- ранжирование результатов;
- агрегация информации;
- аналитика запросов;
- высокая скорость обработки.
- В рамках маркетплейса Elasticsearch используется для:
 - поиска игровых предметов;
 - фильтрации товаров;
 - автодополнения запросов;
 - анализа пользовательской активности;
 - построения статистики.

Использование Elasticsearch позволяет обеспечить мгновенный поиск даже при наличии миллионов записей.

Для визуализации и анализа данных используется Kibana. Данный инструмент позволяет строить графики, отслеживать активность пользователей и анализировать производительность системы.

1.7. Архитектурные особенности высоконагруженных систем

Современные маркетплейсы игровых товаров относятся к категории высоконагруженных распределенных систем. Основными требованиями к таким системам являются:

- высокая отказоустойчивость;
- масштабируемость;
- минимальное время отклика;

- обеспечение безопасности;
- стабильная обработка большого количества запросов.

Для решения данных задач используется микросервисная архитектура. Каждый сервис выполняет отдельную функцию:

- сервис пользователей;
- сервис товаров;
- сервис заказов;
- сервис платежей;
- сервис уведомлений.

Взаимодействие между сервисами осуществляется через брокер сообщений Kafka.

Apache Kafka представляет собой распределенную платформу потоковой передачи данных. Использование Kafka позволяет:

- асинхронно обрабатывать события;
- уменьшить связанность сервисов;
- повысить отказоустойчивость системы;
- обеспечить масштабирование.

Данное событие может быть обработано несколькими сервисами одновременно:

- сервисом уведомлений;
- сервисом аналитики;
- сервисом платежей.

Такой подход обеспечивает гибкость архитектуры и упрощает расширение системы.

1.8. Анализ требований к базе данных маркетплейса игровых товаров

Проектирование базы данных для сервиса по продаже внутриигровых предметов требует предварительного анализа функциональных и

нефункциональных требований системы. От правильности определения требований зависит производительность, масштабируемость и надежность всей информационной системы.

К функциональным требованиям базы данных относятся:

- хранение информации о пользователях;
- хранение каталога игровых товаров;
- обработка заказов и транзакций;
- хранение истории операций;
- поддержка системы отзывов и рейтингов;
- управление пользовательскими балансами;
- поддержка поиска и фильтрации товаров;
- хранение данных о платежах;
- обработка уведомлений и логов.

Каждая из перечисленных функций требует отдельной структуры хранения данных и механизмов обеспечения целостности информации.

Например, система обработки заказов должна гарантировать:

- невозможность двойной покупки одного и того же товара;
- корректное списание денежных средств;
- фиксацию статуса заказа;
- сохранение истории изменений.

Для реализации подобных требований используются транзакции PostgreSQL и ограничения целостности данных.

Нефункциональные требования включают:

- высокую скорость обработки запросов;
- отказоустойчивость;
- масштабируемость;
- безопасность хранения данных;
- возможность горизонтального расширения;
- поддержку резервного копирования.

Высокая нагрузка на маркетплейс игровых товаров обусловлена большим количеством одновременно выполняемых операций. Например, во время распродаж или игровых обновлений количество запросов к системе может увеличиваться в несколько раз. В связи с этим база данных должна быть способна эффективно работать в условиях повышенной нагрузки.

Дополнительно необходимо учитывать требования безопасности. Поскольку система обрабатывает финансовые операции и персональные данные пользователей, необходимо обеспечить:

- шифрование паролей;
- защиту от SQL-инъекций;
- разграничение прав доступа;
- журналирование действий пользователей;
- защиту платежной информации.

Также важным требованием является возможность дальнейшего масштабирования системы без полной переработки архитектуры базы данных.

1.9. Моделирование сущностей и связей базы данных

Одним из важнейших этапов проектирования базы данных является моделирование сущностей предметной области и определение связей между ними.

Для сервиса продажи внутриигровых предметов можно выделить следующие основные сущности:

- пользователь;
- игра;
- категория товара;
- товар;
- заказ;
- платеж;
- баланс;

- отзыв;
- уведомление;
- транзакция.

Каждая сущность содержит определенный набор атрибутов, пример приведен в таблице 2.

Таблица 2 – Описание атрибутов сущности «Товар».

Атрибут	Назначение
1	2
id	Уникальный идентификатор
title	Название товара
description	Описание
price	Стоимость
game_id	Идентификатор игры
seller_id	Идентификатор продавца
status	Статус товара
created_at	Дата публикации

Связи между сущностями могут иметь различные типы:

- один к одному;
- один ко многим;
- многие ко многим.

Например:

- один пользователь может иметь множество заказов;
- одна игра содержит множество товаров;
- один заказ может содержать несколько товаров.

Для реализации связей используются внешние ключи.

Использование внешних ключей позволяет обеспечить ссылочную целостность данных и предотвратить появление некорректных записей.

2. Проектирование и разработка базы данных сервиса по продаже внутриигровых предметов и валюты

2.1. Анализ требований к разрабатываемой базе данных

В рамках дипломной работы разрабатывается база данных для сервиса по продаже внутриигровых предметов и валюты. Основной задачей проектируемой базы данных является обеспечение надежного хранения информации о пользователях, игровых товарах, заказах, платежах и других объектах предметной области, а также предоставление механизмов быстрого доступа к данным при сохранении их целостности и согласованности.

Проектируемая информационная система представляет собой маркетплейс цифровых игровых товаров, в котором пользователи могут размещать объявления о продаже внутриигровых предметов и валюты, осуществлять поиск интересующих товаров, оформлять заказы и выполнять финансовые операции. В связи с этим база данных должна обеспечивать поддержку большого количества взаимосвязанных сущностей и высокую интенсивность операций чтения и записи.

На этапе анализа были выделены основные бизнес-процессы системы:

- регистрация и авторизация пользователей;
- управление пользовательскими профилями;
- публикация игровых товаров;
- поиск и фильтрация предложений;
- оформление заказов;
- проведение платежных операций;
- управление балансом пользователей;
- хранение истории транзакций;
- обработка уведомлений;
- формирование статистических данных.

Для реализации перечисленных процессов база данных должна обеспечивать хранение следующих категорий информации:

- Данные пользователей.
- Информация об играх и категориях товаров.
- Сведения о внутриигровых предметах и валюте.
- Информация о заказах и их статусах.
- Данные о финансовых операциях.
- История действий пользователей.
- Служебные данные системы.

Функциональные требования к базе данных представлены следующим перечнем:

- хранение информации о зарегистрированных пользователях;
- хранение каталога игровых товаров;
- обеспечение возможности добавления новых игр и категорий;
- хранение информации о заказах и платежах;
- ведение истории транзакций;
- поддержка поиска и фильтрации товаров;
- поддержка механизмов авторизации и аутентификации;
- обеспечение возможности масштабирования структуры данных.

Помимо функциональных требований были определены нефункциональные требования.

К требованиям производительности относятся:

- время обработки стандартных запросов не более 100 мс;
- возможность одновременной работы нескольких тысяч пользователей;
- обеспечение быстрого поиска по каталогу товаров;
- минимизация количества обращений к основной базе данных за счет кэширования.

К требованиям надежности относятся:

- обеспечение целостности данных при выполнении транзакций;
- сохранение информации при сбоях оборудования;
- поддержка резервного копирования;

- возможность восстановления данных после аварийных ситуаций.

К требованиям безопасности относятся:

- защита персональных данных пользователей;
- хранение паролей в виде криптографических хэшей;
- разграничение прав доступа;
- защита от SQL-инъекций;
- журналирование критически важных операций.

Особое внимание при проектировании базы данных уделяется обеспечению согласованности финансовых операций. Поскольку система предполагает выполнение платежей и изменение баланса пользователей, все операции должны выполняться в рамках транзакций с соблюдением принципов ACID. Это позволяет исключить потерю данных и возникновение ошибок при одновременном выполнении нескольких операций.

На основании проведенного анализа можно сделать вывод, что разрабатываемая база данных должна обеспечивать надежное хранение большого объема взаимосвязанных данных, поддерживать высокую производительность при интенсивной нагрузке и предоставлять механизмы масштабирования для дальнейшего развития сервиса.

2.2. Проектирование основных сущностей базы данных

Сущность «Пользователь» является центральным элементом разрабатываемой базы данных, поскольку большинство бизнес-процессов сервиса по продаже внутриигровых предметов и валюты связано непосредственно с зарегистрированными пользователями. Через данную сущность осуществляется управление учетными записями, авторизация, проведение финансовых операций, создание заказов и взаимодействие между участниками торговой площадки.

Основным назначением сущности является хранение персональной информации пользователя, параметров его учетной записи, настроек локализации

и сведений, необходимых для функционирования системы безопасности и управления доступом.

Структура сущности «Пользователь» представлена в таблице 3.

Таблица 3 – Основные атрибуты сущности «Пользователь».

Атрибут	Тип данных	Назначение
1	2	3
login	VARCHAR	Уникальное имя пользователя
email	VARCHAR	Адрес электронной почты
password	VARCHAR	Хэш пароля пользователя
phone	VARCHAR	Контактный номер телефона
verified	BOOLEAN	Статус подтверждения электронной почты
verifiedIdentity	BOOLEAN	Статус подтверждения личности
isOnline	BOOLEAN	Признак нахождения пользователя в сети
lastSeen	DATETIME	Дата и время последней активности
avatarKey	VARCHAR	Идентификатор изображения профиля
selectedLocale	ENUM	Выбранный язык интерфейса
selectedCurrency	ENUM	Предпочитаемая валюта отображения цен

Для идентификации пользователей используются уникальные поля login и email. Наличие ограничений уникальности позволяет исключить создание дублирующихся учетных записей и обеспечивает корректную работу механизмов авторизации.

Поле password предназначено для хранения криптографического хэша пароля. Хранение паролей в открытом виде не допускается, поэтому перед сохранением в базу данных пароль проходит процедуру хэширования с использованием современных алгоритмов защиты информации.

В системе реализованы механизмы подтверждения учетной записи и верификации личности пользователя. Для этого используются атрибуты verified и verifiedIdentity. Подтверждение электронной почты позволяет повысить уровень

безопасности учетной записи, а верификация личности может использоваться при совершении финансовых операций или разрешении спорных ситуаций между участниками платформы.

Для отображения актуального статуса пользователя используются поля `isOnline` и `lastSeen`. Данная информация может применяться при организации коммуникации между продавцами и покупателями, а также для отображения активности пользователей в интерфейсе системы.

Особое значение имеют атрибуты `selectedLocale` и `selectedCurrency`. Поскольку сервис ориентирован на работу с пользователями из различных стран, система должна поддерживать многоязычный интерфейс и отображение стоимости товаров в различных валютах. Использование перечислений (ENUM) позволяет ограничить набор допустимых значений и обеспечить целостность данных.

Сущность «Пользователь» имеет несколько связей с другими объектами базы данных.

Для реализации механизма разграничения прав доступа используется связь многие-ко-многим между сущностями «Пользователь» и «Роль». Один пользователь может обладать несколькими ролями, например пользователь, продавец или администратор, а каждая роль может быть назначена множеству пользователей. Данная связь реализуется через промежуточную таблицу `UserRole`.

Также пользователь связан с сущностью финансового счета, историей изменения данных, списком заблокированных пользователей и другими объектами предметной области. Такие связи обеспечивают возможность хранения полной информации о действиях пользователя и его взаимодействии с системой.

Важной особенностью реализации является использование механизма мягкого удаления (Soft Delete), предоставляемого ORM Sequelize посредством параметра `paranoid`. При удалении учетной записи информация физически не удаляется из базы данных, а помечается как удаленная путем заполнения специального поля времени удаления. Такой подход позволяет сохранить историю операций и обеспечивает возможность восстановления учетной записи при необходимости.

Дополнительно в модели реализован механизм каскадного логического удаления связанных записей. При удалении пользователя автоматически выполняется поиск связанных сущностей и их последующее логическое удаление. Это позволяет избежать возникновения «висячих» ссылок и поддерживать согласованность данных во всей системе.

Таким образом, сущность «Пользователь» является фундаментальным элементом разработанной базы данных и обеспечивает хранение информации об участниках маркетплейса, настройках учетных записей, параметрах безопасности и связях с другими объектами предметной области. Реализованная структура позволяет обеспечить масштабируемость системы, поддержку механизмов авторизации и сохранение целостности данных при выполнении операций удаления и изменения информации.

Для обеспечения хранения информации о денежных средствах пользователей в проектируемой базе данных предусмотрена отдельная сущность «Финансовый счет». Выделение финансовой информации в самостоятельную сущность позволяет повысить уровень безопасности системы, упростить сопровождение программного обеспечения и обеспечить возможность дальнейшего расширения функциональности финансового модуля.

В рамках сервиса по продаже внутриигровых предметов и валюты пользователь может осуществлять различные финансовые операции, включая пополнение баланса, оплату заказов, получение средств от продажи товаров и начисление бонусов. В связи с этим возникает необходимость централизованного хранения информации о денежных средствах и их состоянии.

Сущность «Финансовый счет» связана с сущностью «Пользователь» отношением «один к одному». Каждый пользователь обладает собственным финансовым счетом, содержащим сведения о доступных средствах, бонусных начислениях и зарезервированных денежных ресурсах.

Структура сущности представлена в таблице 4.

Таблица 4 – Атрибуты сущности «Финансовый счет».

Атрибут	Тип данных	Назначение
1	2	3
userId	INTEGER	Идентификатор пользователя
rub	DECIMAL(10,2)	Баланс в российских рублях
usd	DECIMAL(10,2)	Баланс в долларах США
eur	DECIMAL(10,2)	Баланс в евро
bonuses	DECIMAL(10,2)	Бонусные средства
hiddenBalanceInRub	DECIMAL(10,2)	Скрытый баланс пользователя
securityDeposit	DECIMAL(10,2)	Гарантийный депозит

Для хранения денежных значений используется тип данных DECIMAL(10,2). Выбор данного типа обусловлен необходимостью выполнения точных финансовых вычислений. Использование типов данных с плавающей точкой может приводить к ошибкам округления, которые недопустимы при обработке денежных операций.

Особенностью реализованной модели является поддержка нескольких валют. Для этого используются отдельные поля rub, usd и eur. Такое решение позволяет хранить фактические остатки денежных средств пользователя в различных валютах без необходимости выполнения постоянной конвертации значений. Подобный подход обеспечивает более высокую скорость обработки финансовых операций и упрощает работу с международными пользователями платформы.

Для реализации программ лояльности предусмотрено поле bonuses, предназначенное для хранения бонусных средств пользователя. Бонусный баланс может использоваться при проведении маркетинговых акций, начислении вознаграждений за активность или предоставлении скидок постоянным клиентам сервиса.

Отдельного внимания заслуживает атрибут securityDeposit, предназначенный для хранения гарантийного депозита. Данный механизм может использоваться для обеспечения безопасности сделок между участниками маркетплейса. Часть

денежных средств временно резервируется системой до завершения сделки, что снижает вероятность мошеннических действий и обеспечивает дополнительную защиту пользователей.

Поле `hiddenBalanceInRub` предназначено для хранения денежных средств, которые временно недоступны для отображения или использования в стандартных операциях. Такой механизм может применяться при обработке спорных ситуаций, проведении финансовых проверок или временной блокировке части средств пользователя.

Выделение финансовой информации в отдельную сущность обеспечивает ряд преимуществ по сравнению с хранением баланса непосредственно в таблице пользователей:

- разделение бизнес-логики пользователей и финансовых операций;
- упрощение сопровождения базы данных;
- повышение безопасности хранения денежных средств;
- возможность масштабирования финансовой подсистемы независимо от других компонентов системы;
- упрощение реализации механизмов аудита и контроля финансовых операций.

Кроме того, данная сущность является центральным элементом финансового модуля системы и выступает связующим звеном между пользователями и сущностью транзакций. Все операции пополнения, списания и перевода денежных средств фиксируются в таблице транзакций и приводят к изменению соответствующих атрибутов финансового счета.

Таким образом, сущность «Финансовый счет» обеспечивает централизованное хранение информации о денежных средствах пользователей, поддерживает работу с несколькими валютами и создает основу для реализации надежной и масштабируемой финансовой подсистемы сервиса по продаже внутриигровых предметов и валюты. Одной из важнейших составляющих финансовой подсистемы сервиса является сущность «Транзакция»,

предназначенная для хранения информации обо всех денежных операциях, выполняемых пользователями платформы. Наличие отдельной таблицы транзакций позволяет обеспечить прозрачность финансовых процессов, организовать аудит операций и реализовать механизмы контроля движения денежных средств.

В процессе функционирования маркетплейса пользователи выполняют различные финансовые действия: пополняют баланс, оплачивают заказы, получают средства от продажи игровых товаров, осуществляют возврат денежных средств или получают бонусные начисления. Каждая подобная операция должна быть зафиксирована в базе данных независимо от её результата и типа.

Для решения данной задачи в структуре базы данных предусмотрена отдельная сущность «Транзакция», связанная с финансовым счетом пользователя отношением «многие к одному». Один финансовый счет может содержать множество транзакций, тогда как каждая транзакция относится только к одному конкретному счету.

Структура сущности представлена в таблице 5.

Таблица 5 – Атрибуты сущности «Транзакция».

Атрибут	Тип данных	Назначение
1	2	3
financeId	INTEGER	Идентификатор финансового счета
amount	DECIMAL(10,2)	Сумма операции
currency	ENUM	Валюта транзакции
type	ENUM	Тип финансовой операции
orderId	VARCHAR	Идентификатор связанного заказа
metadata	JSONB	Дополнительная информация об операции
isHidden	BOOLEAN	Признак скрытой транзакции

Центральным атрибутом сущности является поле amount, содержащее сумму финансовой операции. Для хранения денежных значений используется тип данных

DECIMAL(10,2), обеспечивающий высокую точность вычислений и исключающий ошибки округления, характерные для чисел с плавающей точкой.

Поле `currency` определяет валюту, в которой была выполнена операция. Использование перечисления `ENUM` позволяет ограничить набор допустимых значений и обеспечить целостность данных на уровне базы данных. Такой подход особенно важен для международных сервисов, поддерживающих несколько валют одновременно.

Тип финансовой операции хранится в атрибуте `type`. Использование отдельного классификатора транзакций позволяет разграничить различные виды денежных операций и упростить последующий анализ финансовой активности пользователей. В рамках системы могут использоваться следующие категории операций:

- пополнение баланса;
- списание денежных средств;
- оплата заказа;
- возврат средств;
- начисление бонусов;
- удержание комиссии сервиса;
- перевод между счетами пользователей.

Хранение типа операции в виде перечисления обеспечивает возможность быстрого анализа финансовой активности и формирования отчетности.

Для обеспечения связи финансовых операций с торговыми процессами предусмотрено поле `orderId`, являющееся внешним ключом на сущность заказа. Данная связь позволяет определить, какая именно операция относится к конкретной покупке или продаже товара.

Использование такой связи обеспечивает возможность решения следующих задач:

- построение истории расчетов по заказам;
- проведение финансового аудита;

- разрешение спорных ситуаций между участниками сделки;
- автоматическое формирование отчетности.

Особенностью реализации является использование поля `metadata` с типом данных `JSONB`. Данный механизм позволяет хранить дополнительные сведения о транзакции без изменения структуры таблицы.

В поле `metadata` могут сохраняться:

- идентификаторы внешних платежных систем;
- сведения о комиссии;
- информация о способе оплаты;
- технические данные платежного шлюза;
- служебная информация для внутренней обработки.

Использование `JSONB` обусловлено необходимостью хранения разнородных данных, состав которых может изменяться в зависимости от типа финансовой операции. В отличие от традиционного подхода с созданием множества дополнительных столбцов, применение `JSONB` позволяет сохранить гибкость структуры базы данных и упростить дальнейшее развитие системы.

Для управления отображением финансовых операций используется атрибут `isHidden`. Данный механизм позволяет скрывать отдельные транзакции из пользовательского интерфейса без их физического удаления из базы данных. Подобный подход может применяться при проведении внутренних расчетов, тестировании системы или временном ограничении доступа к отдельным операциям.

Важной особенностью реализации является использование механизма мягкого удаления (`Soft Delete`), реализованного средствами ORM `Sequelize` посредством параметра `paranoid`. При удалении транзакции запись сохраняется в базе данных и помечается специальным признаком удаления. Это позволяет обеспечить сохранность финансовой истории и соответствует требованиям аудита денежных операций.

С точки зрения обеспечения надежности финансовых расчетов сущность «Транзакция» играет ключевую роль в архитектуре системы. Изменение баланса пользователя не производится напрямую без создания соответствующей записи о транзакции. Такой подход позволяет обеспечить полную прослеживаемость движения денежных средств и значительно упрощает поиск ошибок в случае возникновения нештатных ситуаций.

Таким образом, сущность «Транзакция» обеспечивает хранение полной истории финансовых операций пользователей, поддержку различных типов денежных операций и возможность проведения финансового аудита. Использование связей с финансовыми счетами и заказами позволяет интегрировать финансовую подсистему с основными бизнес-процессами маркетплейса и обеспечивает высокий уровень надежности обработки денежных средств.

Центральным элементом торговой подсистемы разработанного сервиса является сущность «Заказ». Данная сущность предназначена для хранения информации о фактах приобретения внутриигровых товаров пользователями платформы и обеспечивает связь между финансовыми операциями, пользователями и объектами продажи.

В рамках функционирования маркетплейса заказ является основным объектом бизнес-процесса купли-продажи. После выбора товара и подтверждения покупки создается запись о заказе, содержащая сведения о приобретаемом товаре, его стоимости, способе оплаты и текущем состоянии сделки.

Структура сущности представлена в таблице 6.

Таблица 6 – Атрибуты сущности «Заказ».

Атрибут	Тип данных	Назначение
1	2	3
id	VARCHAR(8)	Уникальный идентификатор заказа
price	DECIMAL(10,2)	Итоговая стоимость заказа

Продолжение таблицы 6

1	2	3
priceWithoutCommission	DECIMAL(10,2)	Стоимость без учета комиссии сервиса
currency	ENUM	Валюта заказа
currencyForPriceWithoutCommission	ENUM	Валюта исходной стоимости
productLotId	INTEGER	Идентификатор лота товара
productSnapshotId	VARCHAR	Идентификатор снимка товара
buyerId	INTEGER	Идентификатор покупателя
paymentMethod	VARCHAR	Способ оплаты
status	ENUM	Текущий статус заказа
isAlreadyReopened	BOOLEAN	Признак повторного открытия спора
waitToReopen	BOOLEAN	Ожидание повторного открытия
isPriority	BOOLEAN	Приоритетный заказ
isProblematic	BOOLEAN	Признак проблемной сделки

Одной из особенностей реализации является использование короткого строкового идентификатора заказа длиной восемь символов. Генерация идентификаторов выполняется на стороне базы данных посредством специализированной функции PostgreSQL. Такой подход обеспечивает удобство работы пользователей с заказами и упрощает поиск информации по номеру заказа.

Для хранения стоимости заказа используются поля `price` и `priceWithoutCommission`. Разделение этих значений обусловлено необходимостью учета комиссии маркетплейса. Поле `price` содержит итоговую сумму, которую оплачивает покупатель, тогда как `priceWithoutCommission` хранит фактическую стоимость товара без учета сервисных сборов.

Подобное разделение позволяет решать несколько задач:

- рассчитывать доход платформы;
- формировать финансовую отчетность;
- отображать пользователям структуру стоимости заказа;
- выполнять корректные возвраты денежных средств.

Особое значение для маркетплейса игровых товаров имеет поддержка нескольких валют. Для этого используются атрибуты `currency` и `currencyForPriceWithoutCommission`. Такое решение позволяет сохранять информацию о стоимости товара в исходной валюте и одновременно учитывать валюту проведения финансовой операции.

Связь с пользователем реализована через внешний ключ `buyerId`. Каждый заказ принадлежит конкретному покупателю, что позволяет формировать историю покупок и анализировать активность пользователей платформы.

Важной особенностью разработанной модели является использование сущности снимка товара (`OrderProductSnapshot`). Вместо прямого хранения ссылки на текущую версию товара заказ сохраняет ссылку на отдельный снимок состояния товара на момент покупки.

Данный подход позволяет решить одну из ключевых проблем маркетплейсов цифровых товаров. После оформления заказа продавец может изменить описание товара, стоимость или другие характеристики. Использование снимка обеспечивает сохранение первоначального состояния объекта продажи и позволяет восстановить информацию о товаре в том виде, в котором он был приобретен пользователем.

С точки зрения обеспечения юридической и финансовой прозрачности сделки использование снимков товаров является важным механизмом защиты интересов как покупателей, так и продавцов.

Для контроля жизненного цикла сделки используется атрибут `status`. Состояние заказа может изменяться по мере прохождения различных этапов обработки.

Типовой жизненный цикл заказа включает следующие состояния:

- создание заказа;
- ожидание оплаты;
- подтверждение оплаты;
- выполнение заказа;
- завершение сделки;
- отмена заказа;
- открытие спора.

Использование перечисления `ENUM` обеспечивает контроль допустимых состояний и предотвращает возникновение некорректных переходов между статусами.

Дополнительно в модели реализованы специальные признаки, предназначенные для обработки нестандартных ситуаций.

Атрибут `isAlreadyReopened` используется для контроля повторного открытия ранее закрытого спора между участниками сделки. Данное ограничение позволяет предотвратить злоупотребление механизмом арбитража.

Поле `waitToReopen` предназначено для хранения информации о запросе на повторное рассмотрение спорной ситуации.

Для ускоренной обработки отдельных заказов предусмотрен признак `isPriority`. Приоритетные заказы могут обрабатываться системой или модераторами в первую очередь.

Поле `isProblematic` используется для маркировки потенциально проблемных сделок, требующих дополнительного контроля со стороны администрации сервиса.

К таким ситуациям могут относиться подозрительные платежи, жалобы пользователей или обнаружение признаков мошеннических действий.

Таким образом, сущность «Заказ» является центральным элементом торговой подсистемы маркетплейса и обеспечивает хранение полной информации о процессе приобретения внутриигровых товаров. Реализованная структура поддерживает работу с несколькими валютами, механизмами комиссий, системой разрешения споров и сохранением исторического состояния товаров, что позволяет обеспечить надежность и прозрачность проведения торговых операций.

Сущность «Товар» является основным объектом торговой подсистемы сервиса и предназначена для хранения информации о внутриигровых предметах, игровой валюте и других цифровых продуктах, размещаемых пользователями на платформе. Данная сущность выступает связующим элементом между продавцами, покупателями и системой оформления заказов.

В процессе работы маркетплейса каждый пользователь имеет возможность создавать собственные торговые предложения. Для этого в базе данных формируется запись о товаре, содержащая информацию о его стоимости, принадлежности к определенной игре, настройках продажи и дополнительных характеристиках.

Структура сущности представлена в таблице 7.

Таблица 7 – Атрибуты сущности «Товар».

Атрибут	Тип данных	Назначение
1	2	3
gameCategoryId	INTEGER	Категория товара
sellerId	INTEGER	Идентификатор продавца
slug	VARCHAR	Уникальный идентификатор товара
price	DECIMAL(10,2)	Стоимость товара
currency	ENUM	Валюта продажи
autoDelivery	BOOLEAN	Автоматическая выдача товара
active	BOOLEAN	Статус активности товара

1	2	3
deactiveAfterSell	BOOLEAN	Автоматическое отключение после продажи
quantity	INTEGER	Количество доступных единиц
lastLiftingAt	DATETIME	Время последнего поднятия товара

Каждый товар связан с определенной категорией игровой продукции посредством внешнего ключа `gameCategoryId`. Такой подход обеспечивает структурированное хранение каталога и позволяет организовать эффективную систему поиска и фильтрации предложений.

Связь с продавцом реализована через внешний ключ `sellerId`. Каждый товар принадлежит конкретному пользователю платформы, который несет ответственность за корректность предоставленной информации и выполнение обязательств по продаже цифрового продукта.

Для формирования человекочитаемых URL-адресов используется атрибут `slug`. Наличие уникального символьного идентификатора позволяет использовать товар в ссылках, улучшает удобство навигации и способствует повышению качества поисковой оптимизации веб-приложения.

Стоимость товара хранится в поле `price`. Для хранения денежных значений используется тип `DECIMAL(10,2)`, обеспечивающий необходимую точность финансовых вычислений. Валюта продажи определяется атрибутом `currency`, реализованным с использованием перечисления `ENUM`.

Одной из особенностей разработанного сервиса является поддержка автоматической доставки цифровых товаров. Для управления данным механизмом используется поле `autoDelivery`. При включении данного режима передача товара покупателю может осуществляться без участия продавца сразу после подтверждения оплаты заказа. Такой подход позволяет значительно сократить время обработки сделок и повысить удобство использования платформы.

Для управления состоянием торгового предложения применяется атрибут `active`. Только активные товары отображаются в каталоге и доступны для приобретения пользователями. Возможность временного отключения товара позволяет продавцам управлять своими предложениями без необходимости удаления данных из базы.

Дополнительный механизм управления продажами реализован посредством поля `deactiveAfterSell`. После успешного завершения сделки система автоматически переводит товар в неактивное состояние. Данная функция особенно актуальна для уникальных внутриигровых предметов, которые могут быть проданы только один раз.

Поле `quantity` предназначено для хранения количества доступных единиц товара. Наличие данного атрибута позволяет использовать систему не только для продажи уникальных игровых предметов, но и для реализации игровых валют, расходующихся ресурсов и других товаров, допускающих множественные продажи.

В структуре базы данных сущность «Товар» обладает значительным количеством связей с другими объектами предметной области.

Для хранения дополнительных характеристик используются связанные сущности параметров товара (`ProductOption`). Такой подход обеспечивает гибкость структуры данных и позволяет хранить уникальные свойства различных игровых товаров без изменения основной схемы базы данных.

Поддержка многоязычного интерфейса реализуется посредством сущности переводов товара (`ProductTranslation`). Данное решение позволяет хранить описания и названия товаров на нескольких языках, что особенно важно для международных торговых площадок.

Для хранения изображений и других медиафайлов используются связанные записи `ProductFile`. Благодаря этому пользователи получают возможность визуально оценивать предлагаемые игровые товары перед совершением покупки.

Дополнительно система сохраняет статистическую информацию о просмотрах товара через сущность `ProductView`, а также отзывы покупателей

посредством сущности Review. Накопление подобных данных позволяет реализовать механизмы аналитики и оценки качества работы продавцов.

Особое значение имеет связь с сущностью OrderProductSnapshot. При создании заказа система формирует снимок состояния товара, содержащий его характеристики на момент покупки. Такой механизм обеспечивает сохранение истории изменений и позволяет восстановить параметры товара даже после его последующего редактирования продавцом.

Для реализации механизма продажи нескольких экземпляров товара используется сущность ProductLot, содержащая конкретные единицы товара, доступные для передачи покупателю. Подобная архитектура обеспечивает высокую гибкость системы и позволяет поддерживать различные сценарии реализации цифровых продуктов.

Важной особенностью модели является использование механизма мягкого удаления (Soft Delete), реализованного средствами ORM Sequelize. Благодаря этому удаленные товары сохраняются в базе данных и могут использоваться для анализа истории продаж, формирования отчетности и разрешения спорных ситуаций.

Таким образом, сущность «Товар» является ключевым объектом торговой подсистемы маркетплейса и обеспечивает хранение всей необходимой информации о реализуемых цифровых продуктах. Разработанная структура поддерживает гибкую настройку параметров товаров, работу с несколькими языками, автоматическую доставку, сбор статистики и сохранение исторических данных, что позволяет эффективно решать задачи электронной торговли внутриигровыми предметами и валютой.

2.3. Разработка логической модели базы данных

После определения основных сущностей предметной области был выполнен этап разработки логической модели базы данных. Основной целью данного этапа является формирование структуры взаимосвязей между объектами системы,

обеспечивающей целостность данных и поддержку всех бизнес-процессов сервиса по продаже внутриигровых предметов и валюты.

Логическая модель базы данных представляет собой абстрактное описание структуры хранения информации без привязки к конкретной системе управления базами данных. На данном этапе определяются основные сущности, их атрибуты и взаимосвязи, необходимые для реализации функциональных требований системы.

В результате проведенного анализа предметной области были выделены следующие ключевые сущности:

- пользователь (User);
- финансовый счет (Finance);
- транзакция (Transaction);
- товар (Product);
- заказ (Order);
- категория игры (GameCategory);
- лот товара (ProductLot);
- снимок товара (OrderProductSnapshot);
- отзыв (Review);
- роль пользователя (Role).

При проектировании логической модели особое внимание было уделено обеспечению нормализации данных и исключению избыточного хранения информации. Каждая сущность отвечает за хранение строго определенного набора данных, что позволяет минимизировать дублирование информации и упростить сопровождение системы.

Одной из основных связей модели является связь между сущностями «Пользователь» и «Финансовый счет». Для каждого зарегистрированного пользователя создается отдельный финансовый счет, содержащий сведения о денежных средствах и бонусном балансе. Данная связь реализуется по принципу «один к одному».

Сущность «Финансовый счет» связана с сущностью «Транзакция» отношением «один ко многим». Один счет может содержать множество финансовых операций, тогда как каждая транзакция относится только к одному счету. Такое решение обеспечивает возможность хранения полной истории движения денежных средств.

Торговая подсистема построена вокруг сущности «Товар». Каждый товар принадлежит определенному пользователю-продавцу, поэтому между сущностями «Пользователь» и «Товар» существует связь типа «один ко многим». Один продавец может размещать неограниченное количество товаров, однако каждый товар принадлежит только одному владельцу.

Для структурирования каталога товаров используется сущность «Категория игры». Между сущностями «Категория игры» и «Товар» реализована связь «один ко многим», позволяющая группировать предложения по конкретным играм и категориям внутриигровых товаров.

Сущность «Заказ» обеспечивает хранение информации о фактах совершения покупок. Каждый заказ связан с одним покупателем посредством отношения «многие к одному». Один пользователь может оформить множество заказов, однако каждый заказ принадлежит только одному покупателю.

Особенностью разработанной модели является использование сущности «Снимок товара». Вместо хранения прямой ссылки на текущую версию товара заказ содержит ссылку на отдельный снимок его состояния на момент покупки. Данный подход позволяет сохранять историческую информацию о характеристиках товара даже после его изменения продавцом.

Для поддержки механизма множественной продажи игровых ресурсов используется сущность «Лот товара». Один товар может содержать несколько лотов, представляющих собой конкретные экземпляры цифрового продукта, доступные для реализации. Между сущностями «Товар» и «Лот товара» реализована связь «один ко многим».

Дополнительные возможности системы реализуются посредством вспомогательных сущностей. Так, сущность «Отзыв» обеспечивает хранение

пользовательских оценок и комментариев к товарам, а сущность «Роль» используется для реализации механизма разграничения прав доступа. Между сущностями «Пользователь» и «Роль» реализована связь «многие ко многим», что позволяет назначать пользователям несколько ролей одновременно.

При разработке логической модели также были определены основные ограничения целостности данных. Для каждой сущности предусмотрены первичные ключи, обеспечивающие уникальную идентификацию записей. Связи между таблицами реализуются посредством внешних ключей, что позволяет предотвратить появление несогласованных данных и гарантирует корректность взаимосвязей между объектами системы.

Разработанная логическая модель обеспечивает поддержку всех основных бизнес-процессов маркетплейса, включая управление пользователями, размещение товаров, проведение финансовых операций и оформление заказов. Кроме того, структура модели предусматривает возможность дальнейшего расширения функциональности без существенного изменения существующей схемы базы данных.

Таким образом, разработанная логическая модель базы данных формирует основу для последующего построения физической модели и реализации структуры хранения данных в выбранной системе управления базами данных PostgreSQL.

2.4. Разработка физической модели базы данных PostgreSQL

После формирования логической модели базы данных был выполнен этап разработки физической модели, определяющей способ хранения информации в выбранной системе управления базами данных. В качестве основной СУБД проекта была выбрана PostgreSQL, обеспечивающая высокий уровень надежности, поддержку транзакционной обработки данных и широкие возможности масштабирования.

Выбор PostgreSQL обусловлен необходимостью обработки финансовых операций, хранения большого объема взаимосвязанных данных и обеспечения целостности информации. Поскольку сервис по продаже внутриигровых предметов и валюты предполагает выполнение денежных расчетов между пользователями, система управления базами данных должна гарантировать корректность выполнения транзакций и исключать потерю данных при возникновении сбоев.

При разработке физической модели особое внимание уделялось выбору типов данных, обеспечивающих эффективное использование ресурсов и высокую производительность запросов.

Для хранения строковых значений используются типы данных VARCHAR и TEXT. Тип VARCHAR применяется для атрибутов с заранее определенной максимальной длиной, таких как адрес электронной почты, номер телефона или уникальные идентификаторы объектов. Тип TEXT используется для хранения данных переменной длины, объем которых заранее неизвестен.

Для хранения денежных значений во всех финансовых сущностях применяется тип DECIMAL(10,2). Использование данного типа позволяет обеспечить необходимую точность вычислений при работе с денежными средствами и исключить ошибки округления, характерные для чисел с плавающей точкой. Такой подход особенно важен для сущностей «Финансовый счет», «Транзакция» и «Заказ», непосредственно участвующих в обработке платежей.

Для хранения временных меток используются типы DATE и TIMESTAMP. Данные типы применяются для фиксации времени создания объектов, последней активности пользователей, выполнения финансовых операций и других событий системы. Наличие временных меток позволяет организовать аудит действий пользователей и обеспечивает возможность построения аналитической отчетности.

В разработанной базе данных активно используются перечисления (ENUM). Данный механизм применяется для хранения ограниченного набора допустимых значений, таких как:

— валюта финансовой операции;

- статус заказа;
- тип транзакции;
- язык интерфейса пользователя.

Использование ENUM позволяет обеспечить дополнительный контроль целостности данных непосредственно на уровне базы данных и предотвращает сохранение некорректных значений.

Одной из особенностей реализации является использование типа данных JSONB. Данный тип применяется для хранения дополнительной информации о транзакциях и других объектах системы, структура которых может изменяться в процессе развития проекта.

Применение JSONB обеспечивает следующие преимущества:

- хранение динамических данных без изменения структуры таблиц;
- возможность индексации вложенных полей;
- высокая скорость обработки JSON-документов;
- поддержка сложных запросов к вложенным объектам.

Использование JSONB позволило сохранить гибкость модели данных и упростить дальнейшее расширение функциональности системы.

Для обеспечения уникальной идентификации записей в базе данных используются первичные ключи. В большинстве сущностей применяются числовые идентификаторы, автоматически генерируемые системой. Для сущности «Заказ» реализован отдельный механизм генерации коротких строковых идентификаторов, формируемых с использованием пользовательской функции PostgreSQL. Такое решение позволяет повысить удобство работы пользователей с номерами заказов и упрощает взаимодействие со службой поддержки.

Для обеспечения согласованности данных между таблицами используются внешние ключи. Внешние ключи реализуют связи между пользователями, товарами, заказами, финансовыми счетами и транзакциями. Применение данного механизма позволяет предотвратить появление несогласованных записей и гарантирует корректность ссылочной целостности базы данных.

Особое внимание при разработке физической модели было уделено организации индексов. Индексация позволяет существенно сократить время выполнения запросов и повысить производительность системы при работе с большим объемом данных.

Индексы создаются для следующих категорий данных:

- идентификаторы пользователей;
- адреса электронной почты;
- идентификаторы заказов;
- категории товаров;
- статусы заказов;
- валюты финансовых операций.

Использование индексов обеспечивает быстрый поиск записей и снижает нагрузку на сервер базы данных при выполнении операций фильтрации и сортировки.

В разработанной системе также используется механизм мягкого удаления (Soft Delete), реализованный средствами ORM Sequelize посредством параметра `paranoid`. При удалении записи информация физически не удаляется из базы данных, а помечается как удаленная путем заполнения специального поля времени удаления. Такой подход позволяет сохранить историю изменений и обеспечивает возможность восстановления данных при необходимости.

Дополнительно физическая модель предусматривает автоматическое создание временных меток создания и изменения записей. Для большинства сущностей используются поля `createdAt` и `updatedAt`, позволяющие отслеживать жизненный цикл объектов системы и анализировать историю их изменений.

Таким образом, разработанная физическая модель базы данных обеспечивает надежное хранение информации, поддержку финансовых операций, контроль целостности данных и высокую производительность обработки запросов. Использование возможностей PostgreSQL, таких как ENUM, JSONB, индексы и механизмы транзакционной обработки, позволило создать масштабируемую

структуру хранения данных, соответствующую требованиям сервиса по продаже внутриигровых предметов и валюты.

2.5. Реализация моделей базы данных средствами Sequelize ORM

После разработки логической и физической моделей базы данных была выполнена программная реализация структуры хранения данных средствами объектно-реляционного отображения Sequelize ORM. Использование ORM позволяет организовать взаимодействие между приложением и базой данных посредством объектной модели, что значительно упрощает разработку и сопровождение программного обеспечения.

В качестве серверной платформы проекта используется фреймворк NestJS, построенный на языке программирования TypeScript. Для интеграции приложения с PostgreSQL была выбрана библиотека Sequelize, обеспечивающая поддержку объектно-реляционного отображения и предоставляющая удобные средства описания сущностей базы данных.

Выбор Sequelize обусловлен рядом факторов. Во-первых, данная ORM обладает полной поддержкой PostgreSQL и позволяет использовать специфические возможности СУБД, включая типы данных JSONB, ENUM и механизмы транзакционной обработки. Во-вторых, Sequelize предоставляет развитый механизм описания связей между сущностями и поддерживает автоматическое управление схемой базы данных. Кроме того, библиотека хорошо интегрируется с NestJS посредством пакета `sequelize-typescript`, что позволяет использовать возможности TypeScript и декораторов при проектировании моделей.

В разработанной системе каждая таблица базы данных представлена отдельным классом, наследуемым от базового класса `Model`. Такой подход обеспечивает единообразное описание структуры данных и позволяет работать с записями базы данных как с объектами языка программирования.

В качестве примера можно рассмотреть модель пользователя. Для описания таблицы используется декоратор `@Table`, определяющий основные параметры

сущности, включая название таблицы и дополнительные настройки хранения данных.

Описание столбцов выполняется посредством декоратора `@Column`, позволяющего определить тип данных, ограничения целостности и значения по умолчанию. Например, для хранения адреса электронной почты пользователя используется строковый тип данных с ограничением уникальности, что исключает возможность регистрации нескольких учетных записей с одинаковым адресом электронной почты.

Одним из преимуществ Sequelize является поддержка декларативного описания связей между сущностями. В разработанном проекте активно используются основные типы отношений, предусмотренные реляционной моделью данных.

Для реализации связи «один ко многим» применяются декораторы `@HasMany` и `@BelongsTo`. Например, один пользователь может владеть множеством товаров, а каждый товар принадлежит только одному продавцу. Аналогичным образом реализуются связи между финансовыми счетами и транзакциями.

Для реализации связи «многие ко многим» используются декораторы `@BelongsToMany` и промежуточные таблицы. Данный механизм применяется при организации системы ролей пользователей. Один пользователь может обладать несколькими ролями одновременно, а каждая роль может быть назначена множеству пользователей.

Особое значение в проекте имеет использование механизма мягкого удаления данных. Для ряда сущностей, включая пользователей и товары, применяется параметр `paranoid: true`. При удалении объекта соответствующая запись не удаляется физически из базы данных, а помечается специальной временной меткой удаления.

Использование механизма мягкого удаления обеспечивает ряд преимуществ:

- сохранение истории операций;

- возможность восстановления данных;
- предотвращение потери информации при ошибочных действиях пользователей;
- сохранение ссылочной целостности связанных объектов.

Дополнительно в модели пользователя реализован механизм каскадного логического удаления связанных данных. При удалении пользователя система автоматически выполняет поиск зависимых записей и переводит их в состояние логического удаления. Такой подход позволяет избежать появления неконсистентных данных и обеспечивает сохранение целостности всей системы.

Важной особенностью реализации является использование пользовательских типов данных PostgreSQL. В частности, для хранения дополнительной информации о финансовых операциях применяется тип JSONB, позволяющий сохранять структуру данных произвольной сложности без изменения схемы базы данных.

Для обеспечения безопасности финансовых расчетов активно используются встроенные механизмы транзакционной обработки данных. Выполнение операций пополнения баланса, списания денежных средств и создания заказов осуществляется в рамках транзакций базы данных, что гарантирует атомарность выполнения операций и исключает возникновение несогласованных состояний системы.

Дополнительным преимуществом Sequelize является поддержка автоматического управления временными метками. Для большинства сущностей автоматически создаются поля `createdAt` и `updatedAt`, содержащие информацию о времени создания и последнего изменения записи. Наличие данных атрибутов существенно упрощает аудит действий пользователей и анализ истории изменений объектов системы.

В рамках проекта также используются перечисления (ENUM), позволяющие ограничить набор допустимых значений для отдельных атрибутов. Такой подход применяется при хранении валют, статусов заказов, типов транзакций и языковых

настроек пользователей. Использование перечислений позволяет перенести часть контроля целостности данных непосредственно на уровень базы данных.

Реализация моделей посредством Sequelize позволила существенно сократить объем программного кода, необходимого для взаимодействия с базой данных. Вместо ручного формирования SQL-запросов разработчик получает возможность работать с объектами предметной области, что повышает читаемость программного кода и снижает вероятность возникновения ошибок.

Таким образом, применение Sequelize ORM позволило реализовать разработанную структуру базы данных в виде набора взаимосвязанных объектных моделей, обеспечивающих эффективное взаимодействие приложения с PostgreSQL. Использование механизмов объектно-реляционного отображения, поддержки связей, транзакций и мягкого удаления данных обеспечило создание гибкой и масштабируемой системы хранения информации, полностью соответствующей требованиям сервиса по продаже внутриигровых предметов и валюты.

2.6. Реализация кэширования данных с использованием Redis

В разработанном сервисе по продаже внутриигровых предметов и валюты Redis используется как прикладной механизм ускорения доступа к данным, которые наиболее часто запрашиваются пользователями и создают нагрузку на основную базу данных PostgreSQL.

В рамках проекта Redis интегрирован в серверную часть на NestJS и применяется точно для снижения количества повторных запросов к сущностям, которые не требуют строгой транзакционной консистентности.

При отображении списка товаров (Product) система сначала проверяет наличие данных в Redis. Если данные присутствуют — они возвращаются напрямую, без обращения к PostgreSQL.

Если данные отсутствуют:

- выполняется запрос в PostgreSQL;
- результат сериализуется;
- сохраняется в Redis с ограниченным временем жизни.

Это применяется для:

- главной страницы каталога;
- страниц категорий GameCategory;
- списков популярных товаров.

Redis используется для хранения:

- данных текущей сессии пользователя;
- информации о статусе онлайн (isOnline);
- временных данных авторизации.

Это позволяет не обращаться к таблице User при каждом запросе, связанном с проверкой токена или отображением статуса пользователя.

Для сущности Product используется механизм накопления счетчиков:

- просмотры товара (views);
- переходы в карточку;
- добавления в заказ.

Данные сначала записываются в Redis через инкрементирование значений, например через счетчики, и периодически синхронизируются с PostgreSQL.

Это позволяет:

- не перегружать базу частыми UPDATE-запросами;
- формировать рейтинг популярных товаров в реальном времени.

В системе присутствует работа с несколькими валютами (RUB, USD, EUR). Курсы валют и вспомогательные данные (CurrencyValue) кэшируются в Redis, так как:

- они часто запрашиваются при расчете стоимости Order;
- редко изменяются;
- не требуют строгой транзакционной обработки.

Redis в данной системе не заменяет PostgreSQL, а выполняет роль вспомогательного слоя ускорения чтения данных.

Основные задачи Redis в проекте:

- снижение нагрузки на таблицы Product и Order;
- ускорение отображения каталога;
- хранение временных данных пользователя;
- поддержка высоконагруженных операций просмотра и поиска.

Таким образом, Redis используется исключительно для оптимизации операций чтения и не участвует в хранении критически важных финансовых транзакций.

2.7. Реализация системы поиска логов на базе Elasticsearch

В разработанном сервисе по продаже внутриигровых предметов и валюты Elasticsearch используется не для поиска товаров, а для централизованного хранения и анализа логов системы.

Основная причина внедрения Elasticsearch заключается в необходимости обеспечения эффективного поиска и анализа большого объема событий, генерируемых серверной частью приложения на NestJS.

В процессе работы системы формируются различные типы логов, включая: пользовательские действия (регистрация, авторизация, оформление заказа);

- финансовые операции (создание транзакций, изменение баланса);
- изменения сущностей базы данных (Product, Order, Finance);
- системные события и ошибки;
- события безопасности (неуспешные попытки входа, подозрительная активность).

Хранение подобных данных в PostgreSQL не является оптимальным решением, так как реляционная база данных не предназначена для быстрого полнотекстового поиска по большим объемам логов.

В проекте Elasticsearch используется как отдельное хранилище логов, куда передаются события из серверной части приложения.

Каждое лог-событие формируется в виде документа и содержит структурированные данные, такие как:

- тип события;
- временная метка;
- идентификатор пользователя (если есть);
- источник события (модуль системы);
- текстовое описание события;
- уровень важности (info, warning, error, critical).

Логирование в системе реализовано как асинхронный процесс:

В момент выполнения операции (например, создание Order или Transaction) формируется лог-событие.

Событие передается в отдельный слой логирования.

Лог отправляется в Elasticsearch без блокировки основного бизнес-процесса.

Данные индексируются и становятся доступны для поиска.

Такой подход позволяет не влиять на производительность основных операций системы.

3. Тестирование базы данных и разработанного решения

3.1. Организация процесса тестирования и его цели

Тестирование разработанной системы является важным этапом жизненного цикла программного продукта, направленным на подтверждение корректности функционирования базы данных и связанных с ней компонентов серверной части. Поскольку в рамках проекта реализована распределённая архитектура хранения и обработки данных, включающая PostgreSQL, Redis и Elasticsearch, тестирование проводилось комплексно и охватывало как отдельные модули системы, так и их взаимодействие в рамках единого приложения. Основная особенность архитектуры заключается в разделении ответственности между несколькими слоями обработки данных: PostgreSQL используется как основное хранилище бизнес-информации, Redis применяется для ускорения операций чтения за счёт кэширования, а Elasticsearch используется для поиска и анализа логов системы. Взаимодействие с базой данных осуществляется через ORM Sequelize, что обеспечивает единый объектный подход к работе с данными в серверной части, реализованной на NestJS. В связи с этим процесс тестирования был организован с учётом необходимости проверки как отдельных компонентов, так и их совместной работы в составе единой системы.

Целью тестирования являлось подтверждение того, что разработанная база данных и связанные с ней механизмы обеспечивают корректное хранение и обработку данных предметной области, соблюдение целостности данных при выполнении операций, корректную работу транзакционного механизма при финансовых операциях, эффективное использование кэширования, корректную индексацию логов, а также устойчивость системы при параллельной работе пользователей. Достижение данной цели позволяет подтвердить готовность системы к эксплуатации в условиях высокой нагрузки, характерной для маркетплейса внутриигровых товаров и валюты.

В рамках тестирования были определены следующие основные задачи:

- проверка корректности структуры таблиц и связей между сущностями базы данных;
- проверка ограничений целостности, включая первичные и внешние ключи;
- проверка уникальности данных и ограничений ENUM-типов;
- тестирование операций создания, обновления и удаления данных;
- проверка механизма мягкого удаления (soft delete);
- тестирование транзакционной обработки финансовых операций;
- оценка эффективности кэширования данных в Redis;
- проверка корректности записи и поиска логов в Elasticsearch;
- анализ поведения системы при увеличении количества параллельных запросов.

— Подход к проведению тестирования включал использование двух уровней проверки. На уровне модульного тестирования проверялись отдельные компоненты системы, включая модели Sequelize, логику транзакций, работу с финансовыми операциями и механизм кэширования Redis. На уровне интеграционного тестирования проверялось взаимодействие между основными компонентами системы, включая PostgreSQL, Redis и Elasticsearch, в рамках реальных пользовательских сценариев, таких как просмотр товаров, оформление заказов и обработка финансовых операций.

— Среда тестирования была максимально приближена к условиям реальной эксплуатации системы и включала следующие компоненты:

- серверная часть, реализованная на NestJS;
- система управления базами данных PostgreSQL;
- ORM Sequelize для взаимодействия с базой данных;
- кэширующий слой Redis;
- система логирования и анализа Elasticsearch;
- контейнеризация с использованием Docker (при необходимости развёртывания).

Таким образом, тестирование охватывает все ключевые уровни архитектуры системы и позволяет оценить корректность работы базы данных, устойчивость транзакционных операций и эффективность взаимодействия между компонентами распределённой системы хранения и обработки данных.

3.2. Тестирование реляционной модели данных и целостности PostgreSQL

Тестирование реляционной модели данных и целостности PostgreSQL было направлено на проверку корректности реализации структуры базы данных, созданной на основе моделей Sequelize, а также на подтверждение соблюдения ограничений целостности и корректности связей между сущностями. В рамках данного этапа проверялась работа основных таблиц системы, включая User, Product, Order, Finance, Transaction и связанные с ними вспомогательные сущности. Особое внимание уделялось корректному созданию внешних ключей, обеспечивающих связь между пользователями, товарами, заказами и финансовыми операциями.

В ходе тестирования было проверено формирование связей между сущностями в соответствии с логической моделью. Установлено, что каждый пользователь может иметь связанные записи в таблицах товаров и финансового счета, при этом каждый товар корректно связан с конкретным продавцом, а каждый заказ связан с покупателем и снимком товара на момент покупки. Также подтверждено, что финансовый счет пользователя связан с транзакциями по принципу один ко многим, что обеспечивает корректное хранение истории всех финансовых операций.

Отдельное внимание уделялось проверке ограничений целостности данных. Были протестированы следующие аспекты:

- уникальность идентификаторов пользователей и товаров;
- уникальность электронной почты пользователя;

- корректность работы ENUM-типов для валют, статусов заказов и типов транзакций;
- обязательность заполнения ключевых полей;
- корректность работы внешних ключей при вставке и удалении данных.

В ходе тестирования установлено, что при попытке вставки некорректных значений (например, несуществующих значений ENUM или ссылок на несуществующие записи) база данных корректно отклоняет операции, тем самым предотвращая нарушение целостности данных.

Также был проверен механизм мягкого удаления (soft delete), реализованный через параметр `paranoid` в Sequelize. В результате тестирования подтверждено, что при удалении записей данные физически не удаляются из базы данных, а помечаются как удалённые с помощью временной метки. При этом связанные сущности сохраняют корректную ссылочную структуру и не теряют целостности.

Дополнительно проводилось тестирование генерации идентификаторов для сущности `Order`. Установлено, что пользовательская функция генерации идентификаторов обеспечивает уникальность значений и корректное создание коротких строковых ID без возникновения коллизий при одновременном создании нескольких заказов.

В целом результаты тестирования показали, что реляционная модель данных реализована корректно, а PostgreSQL обеспечивает надёжное хранение информации и поддержание целостности данных в рамках всех ключевых бизнес-сценариев системы.

3.3. Тестирование корректности и атомарности финансовых операций

Тестирование транзакционной подсистемы, включающей сущности `Order`, `Finance` и `Transaction`, выполнялось с целью проверки корректности обработки финансовых операций, а также обеспечения атомарности всех критически важных изменений данных. Данная подсистема является ключевой частью разработанного

маркетплейса, поскольку именно она отвечает за списание и зачисление средств при совершении покупок внутриигровых товаров.

В рамках тестирования были проверены основные сценарии финансового взаимодействия пользователей с системой. К ним относятся создание заказа с автоматическим списанием средств с баланса пользователя, формирование записи о транзакции, пополнение баланса пользователя, а также корректная обработка операций в разных валютах. Дополнительно проверялись сценарии одновременного выполнения нескольких финансовых операций разными пользователями.

Особое внимание уделялось обеспечению атомарности операций. Все финансовые действия выполнялись в рамках транзакций PostgreSQL, что позволило гарантировать выполнение принципа “всё или ничего”. Это означает, что при создании заказа одновременно выполняются несколько операций: изменение баланса в таблице Finance, создание записи в таблице Order и фиксация операции в таблице Transaction. В случае возникновения ошибки на любом этапе выполняется откат всей транзакции, что исключает возможность частичного изменения данных.

В процессе тестирования также проверялась работа системы при конкурентном доступе к одному и тому же финансовому счёту. Были смоделированы ситуации одновременного списания средств с одного баланса несколькими запросами. Результаты показали, что PostgreSQL корректно обрабатывает блокировки и предотвращает возникновение гонок данных, обеспечивая согласованность состояния финансового счёта пользователя.

Дополнительно тестировалась корректность формирования записей в таблице Transaction. Проверялось, что каждая финансовая операция сопровождается созданием соответствующей записи, содержащей сумму, тип операции, валюту и привязку к финансовому счёту пользователя. Это обеспечивает возможность последующего аудита всех финансовых действий в системе.

Также был проверен сценарий работы с различными валютами, поддерживаемыми системой. Установлено, что при выполнении операций

корректно учитывается валюта заказа и осуществляется правильное сопоставление с балансом пользователя, что исключает ошибки при конвертации и расчётах.

В результате проведённого тестирования установлено, что финансовая подсистема функционирует корректно, обеспечивает целостность данных и устойчиво работает в условиях конкурентного доступа. Использование транзакционного механизма PostgreSQL позволяет гарантировать надёжность выполнения всех операций, связанных с изменением финансового состояния пользователей.

3.4. Тестирование Redis (кэширования данных)

Тестирование механизма кэширования Redis было направлено на оценку эффективности ускорения операций чтения данных и снижения нагрузки на основную базу данных PostgreSQL. В рамках разработанного сервиса Redis используется как промежуточный слой хранения часто запрашиваемой информации, что позволяет минимизировать количество обращений к реляционной базе данных и повысить общую производительность системы.

В процессе тестирования проверялись сценарии получения данных, которые наиболее часто используются в системе. К ним относятся загрузка списка товаров, получение информации о конкретном пользователе, а также обращение к данным, связанным с валютами и расчетами стоимости товаров. В каждом из этих сценариев анализировалось время ответа системы при первом запросе и при повторных обращениях к тем же данным.

Установлено, что при первом запросе данные извлекаются из PostgreSQL, после чего результат сохраняется в Redis. При последующих запросах данные возвращаются непосредственно из кэша без обращения к основной базе данных, что существенно сокращает время отклика системы. Данный механизм особенно

эффективно проявляет себя при работе с каталогом товаров, где наблюдается высокая частота повторных запросов от пользователей.

Отдельно проверялась стратегия актуализации данных в кэше. В системе реализован механизм ограничения времени жизни записей (TTL), благодаря которому устаревшие данные автоматически удаляются из Redis. Для различных типов данных используется разное время хранения, в зависимости от их критичности и частоты обновления. Например, данные сессий пользователей имеют более короткий срок жизни, тогда как информация о товарах может кэшироваться на более длительный период.

Также тестировался механизм инвалидации кэша при изменении данных в основной базе. В случае обновления информации о товаре или изменении его параметров соответствующие записи в Redis удаляются или обновляются, что предотвращает отображение устаревших данных пользователям.

В ходе тестирования было установлено, что использование Redis позволяет значительно снизить нагрузку на PostgreSQL при повторных запросах и повысить скорость обработки пользовательских операций, связанных с чтением данных. Особенно заметный эффект наблюдается при работе с часто запрашиваемыми сущностями, такими как товары и данные профиля пользователя.

Таким образом, тестирование подтвердило корректность реализации механизма кэширования и его положительное влияние на производительность системы в условиях высокой нагрузки.

Заключение

В ходе выполнения дипломной работы была разработана и реализована база данных для сервиса по продаже внутриигровых предметов и валюты. Актуальность данной темы обусловлена активным развитием игровой индустрии и увеличением числа цифровых торговых площадок, обеспечивающих обмен виртуальными товарами между пользователями. Подобные системы требуют высокой надежности хранения данных, строгого соблюдения целостности информации, а также способности обрабатывать большое количество транзакций в условиях высокой нагрузки.

Основной целью работы являлось проектирование и реализация базы данных, обеспечивающей корректное функционирование маркетплейса внутриигровых товаров, включая управление пользователями, товарами, заказами и финансовыми операциями. В процессе выполнения работы цель была полностью достигнута за счет последовательного решения поставленных задач, начиная от анализа предметной области и заканчивая тестированием разработанной системы.

На этапе анализа предметной области была изучена структура современных торговых платформ в игровой индустрии, выявлены основные сущности и их взаимосвязи. Особое внимание уделялось специфике работы с цифровыми товарами, которые обладают динамической стоимостью, зависящей от внутриигровых факторов, а также требуют учета различных типов операций, включая продажу предметов, передачу внутриигровой валюты и выполнение услуг между пользователями.

На основе проведенного анализа была разработана логическая модель базы данных, включающая основные сущности системы: пользователей, товары, заказы, финансовые счета и транзакции. При проектировании структуры данных учитывались требования к целостности, масштабируемости и производительности системы. В частности, была предусмотрена возможность расширения

функциональности без необходимости кардинального изменения структуры базы данных.

Физическая реализация базы данных была выполнена на основе PostgreSQL, который был выбран благодаря своей надежности, поддержке транзакций, расширенным возможностям работы с индексами и поддержке сложных типов данных, включая JSONB. Использование данного типа данных позволило эффективно хранить гибкие атрибуты предметов и расширяемые характеристики товаров без нарушения нормализации структуры.

Для взаимодействия серверной части приложения с базой данных был использован ORM Sequelize, интегрированный в серверное приложение на NestJS. Данный подход позволил упростить работу с SQL-запросами, обеспечить единообразную структуру работы с данными и повысить читаемость и сопровождаемость кода. При этом были учтены потенциальные ограничения ORM-подхода, включая возможные избыточные запросы, что частично компенсировалось оптимизацией структуры запросов и использованием индексов на уровне базы данных.

В рамках архитектуры системы также были реализованы дополнительные компоненты хранения и обработки данных. Для ускорения операций чтения был внедрен Redis, используемый в качестве кэширующего слоя. Его применение позволило значительно снизить нагрузку на основную базу данных за счет хранения часто запрашиваемых данных, таких как список товаров, информация о пользователях и вспомогательные справочные данные. Использование механизма TTL и инвалидации кэша обеспечило актуальность данных и предотвратило отображение устаревшей информации.

Отдельным компонентом системы является Elasticsearch, применяемый для хранения и анализа логов. Его использование обусловлено необходимостью обработки большого объема событий, генерируемых системой в процессе работы. Elasticsearch обеспечивает возможность быстрого поиска по логам, фильтрации событий по различным параметрам, а также анализа поведения пользователей и

состояния системы. Это существенно повышает наблюдаемость системы и упрощает диагностику ошибок.

Особое внимание в рамках разработки было уделено финансовой подсистеме, включающей сущности Order, Finance и Transaction. Данная подсистема реализует механизм обработки платежей, списания средств и фиксации финансовых операций. Все критические операции выполняются в рамках транзакций PostgreSQL, что обеспечивает атомарность изменений и исключает возможность частичного обновления данных. Это является ключевым требованием для систем, связанных с финансовыми операциями.

В ходе реализации также была предусмотрена поддержка конкурентного доступа к данным, что особенно важно для маркетплейса с большим количеством одновременно работающих пользователей. Проведенные проверки показали, что база данных корректно обрабатывает параллельные запросы, предотвращая возникновение конфликтов и обеспечивая согласованность данных.

Завершающим этапом работы стало тестирование разработанной системы. В рамках тестирования были проверены ключевые аспекты функционирования базы данных, включая корректность структуры таблиц, соблюдение ограничений целостности, работу транзакций, эффективность кэширования и корректность логирования событий. Результаты тестирования подтвердили, что все компоненты системы функционируют стабильно и соответствуют поставленным требованиям.

Таким образом, разработанная база данных представляет собой устойчивое и масштабируемое решение, способное обеспечить работу сервиса по продаже внутриигровых предметов и валюты в условиях высокой нагрузки и интенсивного роста объема данных. Практическая значимость работы заключается в возможности использования предложенной архитектуры в реальных проектах, связанных с цифровыми маркетплейсами и игровыми платформами.

Выполненная работа демонстрирует успешную реализацию всех поставленных задач, включая проектирование структуры данных, выбор технологий хранения и обработки информации, реализацию серверной логики взаимодействия с базой данных и проведение комплексного тестирования системы.

Полученные результаты подтверждают корректность выбранных архитектурных решений и их применимость в условиях реальных высоконагруженных систем.

Список использованных источников

1. Date C. J. Введение в системы баз данных. — 8-е изд. — Бостон: Addison-Wesley, 2019. — 1024 с.
2. Silberschatz A., Korth H. F., Sudarshan S. Концепции систем баз данных. — 7-е изд. — Нью-Йорк: McGraw-Hill, 2020. — 1376 с.
3. Kleppmann M. Проектирование систем обработки больших объёмов данных. — Севастополь: O'Reilly Media, 2017. — 616 с.
4. Fowler M. Архитектурные шаблоны корпоративных приложений. — Бостон: Addison-Wesley, 2002. — 560 с.
5. ISO/IEC 9075:2023 Информационные технологии — Язык баз данных SQL.
6. Martin R. C. Чистая архитектура: руководство по структуре и дизайну программных систем. — Бостон: Pearson, 2017. — 432 с.
7. PostgreSQL Global Development Group. Документация PostgreSQL [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/>
8. Redis Ltd. Документация Redis [Электронный ресурс]. — URL: <https://redis.io/docs/>
9. Elasticsearch B.V. Документация Elasticsearch [Электронный ресурс]. — URL: <https://www.elastic.co/guide/>
10. Команда Sequelize. Документация Sequelize ORM [Электронный ресурс]. — URL: <https://sequelize.org/>
11. Команда NestJS. Официальная документация NestJS [Электронный ресурс]. — URL: <https://docs.nestjs.com/>
12. Docker Inc. Документация Docker [Электронный ресурс]. — URL: <https://docs.docker.com/>
13. Apache Kafka. Документация Apache Kafka [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/>

14. Microsoft TypeScript Team. Документация TypeScript [Электронный ресурс]. — URL: <https://www.typescriptlang.org/docs/>
15. Мартин Фаулер. Микросервисы [Электронный ресурс]. — URL: <https://martinfowler.com/articles/microservices.html>
16. Хабр — статьи по архитектуре и разработке программных систем [Электронный ресурс]. — URL: <https://habr.com/ru/>
17. PostgreSQL Documentation: тип JSON и JSONB [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/current/datatype-json.html>
18. Теорема CAP в распределённых системах [Электронный ресурс]. — URL: https://ru.wikipedia.org/wiki/Теорема_CAP

Приложение 1. Репозиторий



Ссылка для ручного ввода:

<https://github.com/Aleksei835/vkr>

